

- RAG Learning

2.3 Algorithms

Computer Science — OCR — Component 02 (Algorithms and Programming)

This is one of the **largest and most heavily examined** topics on H446. You must be able to *describe, trace, and analyse* every algorithm on this list, as well as compare them using **big-O notation**. Long-answer questions frequently require you to write pseudocode, trace a worked example, and justify a choice of algorithm for a given scenario.

2.3.1 Algorithm Analysis and Design

What is an Algorithm?

An **algorithm** is a finite, ordered sequence of unambiguous, executable instructions that transforms one or more inputs into one or more well-defined outputs in a finite amount of time.

The five essential properties of a formal algorithm are:

- **Finiteness** — the algorithm must terminate after a finite number of steps. An infinite loop is therefore *not* an algorithm.
- **Definiteness (determinism)** — each step must be precisely and unambiguously defined; given the same input it produces the same output.
- **Clearly defined inputs** — the algorithm takes zero or more specified inputs drawn from a specified set.
- **Clearly defined outputs** — it produces one or more outputs that have a specified relationship to the inputs.
- **Effectiveness** — each step must be basic enough that it can in principle be carried out exactly by a person using paper and pencil.

Exam Tip: In a "define algorithm" question, do *not* just write "a set of steps". Mention **finite**, **unambiguous**, and **inputs produce outputs** for full marks.

Why Analyse Algorithms?

Two algorithms may both solve the same problem correctly, but have radically different **performance**. Analysing algorithms lets us:

- Predict how running time and memory usage will scale as the input size grows.
- Compare competing algorithms *before* implementing them.
- Identify bottlenecks and guarantees for worst-case behaviour (critical in real-time, medical or safety-critical systems).
- Choose the right algorithm for the constraints of a particular platform (embedded, mobile, server).

There are two resources we usually measure:

- **Time complexity** — how the number of basic operations grows with input size n .
- **Space complexity** — how the memory used grows with input size n , excluding the input itself (i.e. the *auxiliary* space).

We do *not* measure raw wall-clock time, because that depends on the CPU, language, compiler and competing load. Counting operations and expressing the count as a function of n gives a platform-independent comparison.

Big-O Notation

Big-O notation describes an **asymptotic upper bound** on the growth of a function. We say $f(n) = O(g(n))$ if there exist constants $c > 0$ and $n_0 > 0$ such that $0 \leq f(n) \leq c \cdot g(n)$ for all $n \geq n_0$. Intuitively, $g(n)$ is a ceiling that $f(n)$ never rises above (up to a constant factor) once n is large enough.

Two key simplifications follow from this definition:

- **Drop constants:** $O(3n) = O(n)$; $O(n/2) = O(n)$. Constant factors reflect hardware, not the algorithm.
- **Keep only the dominant term:** $O(n^2 + 5n + 9) = O(n^2)$. For large n the highest-order term overwhelms everything else.

We care primarily about the **worst case** because it provides a *guarantee*: no matter what input arrives, the algorithm will never be slower than this. Average-case analysis is also used but usually requires assumptions about input distribution that may not hold in practice.

Exam Tip: Big-O is an *upper* bound; Big-Ω (Omega) is a *lower* bound; Big-Θ (Theta) is a tight bound. OCR questions use Big-O, but knowing it is specifically the *worst-case upper bound* will stop you losing marks for saying "the algorithm runs in $O(n)$ " when you mean on average.

Complexity Classes

The following table lists, in order of increasing growth, the eight complexity classes you must recognise.

Class	Name	Characteristic behaviour	Example algorithms
$O(1)$	Constant	Runtime does not depend on n at all. Doubling n has no effect.	Array index lookup $a[i]$, hash-table insert/lookup (average), pushing onto a stack.
$O(\log n)$	Logarithmic	Runtime grows by a fixed amount every time n doubles. Problem size halves each step.	Binary search on a sorted array, balanced BST lookup / insert, binary-heap insert.
$O(n)$	Linear	Runtime grows proportionally to n . Doubling n doubles the time.	Linear search, computing a sum, finding max in unsorted array, a single pass through a list.
$O(n \log n)$	Linearithmic	Slightly worse than linear; the provable lower bound for comparison-based sorting.	Merge sort, heap sort, average-case quick sort, FFT.

$O(n^2)$	Quadratic	Two nested loops over n . Doubling n quadruples the time.	Bubble sort, insertion sort, selection sort, simple nested-loop duplicate check.
$O(n^3)$	Cubic	Three nested loops over n .	Naive matrix multiplication (for $n \times n$ matrices), Floyd–Warshall all-pairs shortest paths.
$O(2^n)$	Exponential	Runtime doubles for each extra element. Infeasible beyond roughly $n = 30$.	Naive recursive Fibonacci, brute-force subset sum, brute-force satisfiability (SAT).
$O(n!)$	Factorial	Grows faster than any exponential. Infeasible beyond roughly $n = 12$.	Brute-force travelling salesman, generating all permutations of a list.

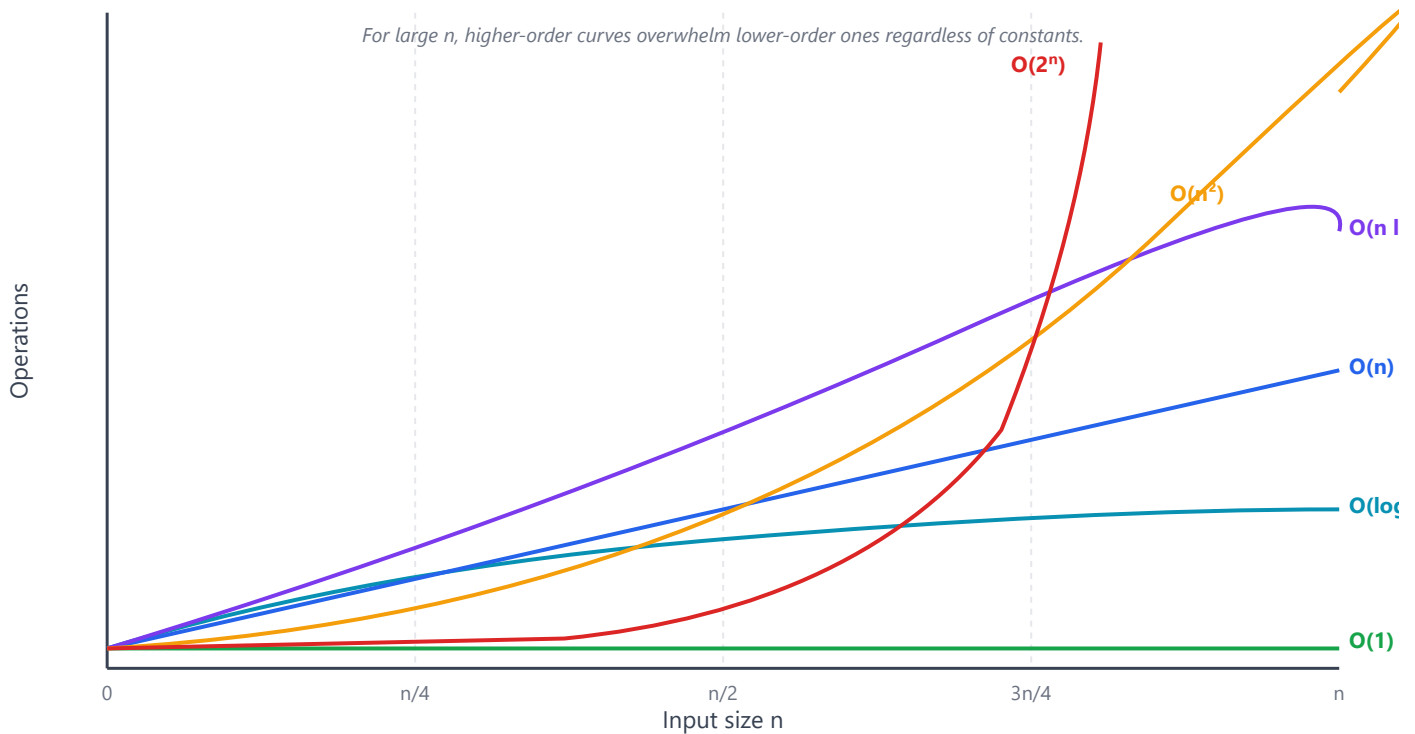
To make the numbers concrete, suppose each operation takes 1 nanosecond. For $n = 60$:

Complexity	Operations	Time at 1 ns each
$O(1)$	1	~1 ns
$O(\log n)$	~6	~6 ns
$O(n)$	60	60 ns
$O(n \log n)$	~355	~355 ns
$O(n^2)$	3 600	3.6 μ s
$O(n^3)$	216 000	0.22 ms
$O(2^n)$	$\sim 1.15 \times 10^{18}$	~36 years
$O(n!)$	$\sim 8.3 \times 10^{81}$	longer than the age of the universe many times over

Exam Tip: Learn the ordering: $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$. OCR examiners frequently ask candidates to rank algorithms by efficiency.

Growth Comparison Graph

Comparison of Big-O Growth Rates



Best, Average and Worst Case

The running time of most algorithms depends not just on the *size* of the input but also on its *shape*. We therefore distinguish three cases:

- **Best case** — the input arrangement that makes the algorithm finish fastest (e.g. target at position 0 for linear search).
- **Average case** — the expected running time over all "typical" inputs. Often requires a probabilistic assumption.
- **Worst case** — the input that maximises running time (e.g. target not in the list at all for linear search).

A classic example where the three differ dramatically is **quick sort**:

Case	Input shape	Complexity
Best	Pivot always splits list roughly in half	$O(n \log n)$
Average	Random input, random or median-of-three pivot	$O(n \log n)$
Worst	Already sorted (or reverse-sorted) list with first-element pivot	$O(n^2)$

Exam Tip: When asked "why is bubble sort best-case $O(n)$?", your answer must mention the **early-termination flag**: if no swap happens on a pass, the list is sorted and the algorithm terminates after one linear pass.

Space Complexity

Space complexity measures the amount of auxiliary memory an algorithm uses as a function of the input size n , excluding the memory required for the input itself.

Two useful distinctions:

- **In-place** algorithms use $O(1)$ or $O(\log n)$ extra memory — they rearrange data within the original array without needing a large second copy. Quick sort and insertion sort are in-place.
- **Out-of-place** algorithms need $O(n)$ or more auxiliary space. Merge sort is out-of-place because merging two halves requires a temporary buffer of length n .

On memory-constrained devices (microcontrollers, mobile, embedded) a slightly slower in-place algorithm is often preferred over a faster out-of-place one.

Analysing a Simple Code Fragment

To practise extracting a big- O from code, count the number of times the innermost statement executes as a function of n .

```
FOR i := 0 TO n - 1           // outer loop runs n times
  FOR j := 0 TO n - 1         // inner loop runs n times
    x := x + 1                 // executes n * n = n² times
```

```
ENDFOR  
ENDFOR
```

Total work: $n \times n = n^2$, hence $O(n^2)$.

```
i := 1  
WHILE i < n  
    print(i)  
    i := i * 2           // i doubles each pass  
ENDWHILE
```

The value of `i` goes 1, 2, 4, 8, ..., and the loop exits when `i ≥ n`. That takes $\log_2 n$ iterations, giving $O(\log n)$.

```
FOR i := 1 TO n  
    j := 1  
    WHILE j < n  
        j := j * 2  
    ENDWHILE  
ENDFOR
```

Outer loop n times, inner loop $\log n$ times per iteration, so $O(n \log n)$ overall. This pattern is exactly what happens in merge sort and heap sort.

Exam Tip: When asked to "find the complexity" of a code fragment, state clearly: (1) identify the innermost statement; (2) count how many times it runs as a function of `n`; (3) simplify (drop constants and lower-order terms).

2.3.2 Searching Algorithms

Linear Search

Linear (sequential) search examines each element of a list in turn from the start until either

the target value is found or the end of the list is reached.

Linear search works on any list — the list does *not* need to be sorted. This is its only real advantage.

```
FUNCTION linearSearch(list, target)
  FOR i := 0 TO length(list) - 1
    IF list[i] = target THEN
      RETURN i                // found – return index
    ENDIF
  ENDFOR
  RETURN -1                  // not found
ENDFUNCTION
```

Worked example. Searching for 17 in [4, 22, 17, 11, 8] :

1. $i=0$: compare 4 vs 17 — no match.
2. $i=1$: compare 22 vs 17 — no match.
3. $i=2$: compare 17 vs 17 — match! Return 2 .

Case	Input description	Complexity
Best	Target is the first element	$O(1)$
Average	Target equally likely at any position	$O(n)$
Worst	Target is last or not present	$O(n)$
Space	Uses a single index variable	$O(1)$

When to use: unsorted data; very short lists where the constant overhead of binary search is not worth it; data held in a structure that does not support random access (e.g. a linked list).

Linear search on a linked list

Because a linked list does not provide $O(1)$ random access, binary search cannot be used on it directly — finding the middle element takes $O(n)$ in itself. For linked lists, linear search is therefore the default:

```
FUNCTION linearSearchLinkedList(head, target)
    current := head
    position := 0
    WHILE current != NULL
        IF current.value = target THEN
            RETURN position
        ENDIF
        current := current.next
        position := position + 1
    ENDWHILE
    RETURN -1
ENDFUNCTION
```

Average performance of linear search (expected number of comparisons) is about $n / 2$ when the target is present and uniformly distributed, which still simplifies to $O(n)$.

Binary Search

Binary search finds a value in a **sorted** list by repeatedly halving the search space. At each step the middle element is compared to the target; the half that cannot contain the target is discarded.

Exam Tip: Binary search **requires sorted data**. Every long-answer question must state this assumption. If the data is not sorted, you must sort it first (costing $O(n \log n)$) or use linear search.

Three pointers are maintained: `low`, `high`, and the midpoint `mid = (low + high) DIV 2`.

Iterative version

```
FUNCTION binarySearchIter(list, target)
    low := 0
    high := length(list) - 1
    WHILE low <= high
        mid := (low + high) DIV 2
        IF list[mid] = target THEN
            RETURN mid
        ELSEIF list[mid] < target THEN
```

```

        low := mid + 1
    ELSE
        high := mid - 1
    ENDIF
ENDWHILE
RETURN -1
ENDFUNCTION

```

Recursive version

```

FUNCTION binarySearchRec(list, target, low, high)
    IF low > high THEN
        RETURN -1 // base case – not found
    ENDIF
    mid := (low + high) DIV 2
    IF list[mid] = target THEN
        RETURN mid
    ELSEIF list[mid] < target THEN
        RETURN binarySearchRec(list, target, mid + 1, high)
    ELSE
        RETURN binarySearchRec(list, target, low, mid - 1)
    ENDIF
ENDFUNCTION

```

Why $O(\log n)$?

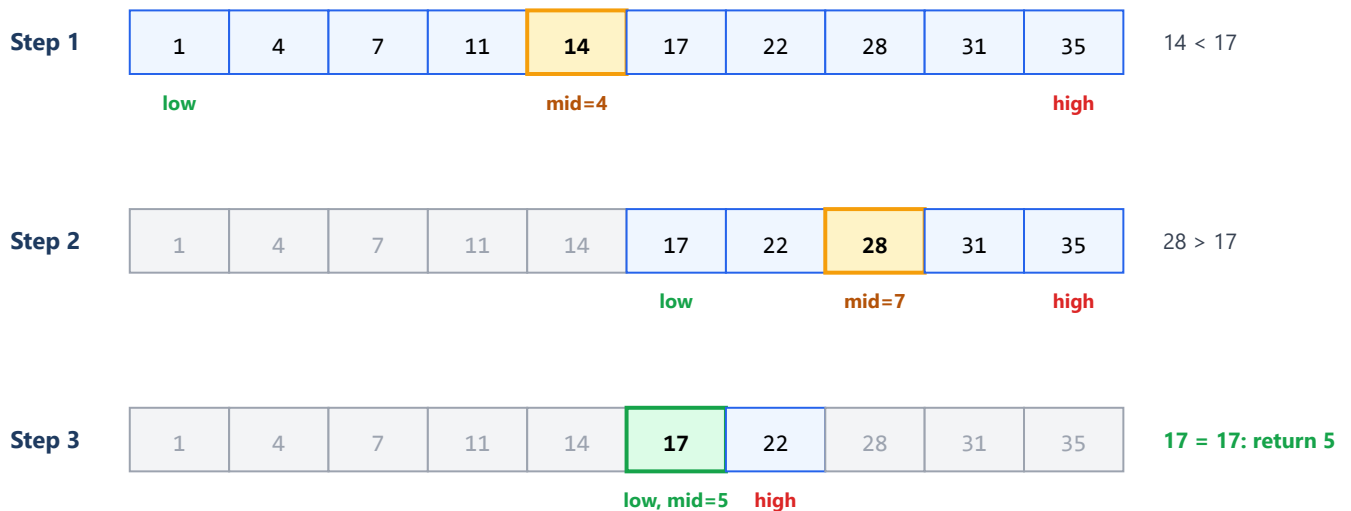
After 1 comparison we have $n/2$ elements left, after 2 we have $n/4$, after k comparisons we have $n/2^k$. The search finishes when only one element remains: $n/2^k = 1$, i.e. $k = \log_2 n$. Doubling the list size adds only **one more comparison**.

Worked example — find 17 in [1, 4, 7, 11, 14, 17, 22, 28, 31, 35]

Step	low	high	mid	list[mid]	Comparison	Action
1	0	9	4	14	$14 < 17$	$low := 5$
2	5	9	7	28	$28 > 17$	$high := 6$
3	5	6	5	17	$17 = 17$	return 5

Found after just 3 comparisons. Linear search would have needed 6.

Binary Search: find 17 in sorted array



Each step halves the remaining search space $\rightarrow \log_2(10) \approx 3$ comparisons

Case	Complexity
Best (target at mid)	$O(1)$
Average	$O(\log n)$
Worst (target absent)	$O(\log n)$
Space (iterative)	$O(1)$
Space (recursive)	$O(\log n)$ call stack

Binary Tree Search

A **binary search tree (BST)** is a binary tree with the property that for every node, all values in the left subtree are smaller and all values in the right subtree are larger. Searching a BST resembles binary search but on a linked structure rather than an array.

```

FUNCTION bstSearch(node, target)
  IF node = NULL THEN
    RETURN NULL           // not found
  ENDIF
  IF target = node.value THEN

```

```

    RETURN node
ELSEIF target < node.value THEN
    RETURN bstSearch(node.left, target)
ELSE
    RETURN bstSearch(node.right, target)
ENDIF
ENDFUNCTION

```

Case	Tree shape	Complexity
Best / Average	Balanced tree (height $\approx \log n$)	$O(\log n)$
Worst	Degenerate tree — values inserted in sorted order, height = n (effectively a linked list)	$O(n)$

Exam Tip: Explain **why** the worst case is $O(n)$: if nodes are inserted in already-sorted order (e.g. 1, 2, 3, 4, ...), every insertion goes right, producing a tree of height n where search degenerates to linear search. Self-balancing trees (AVL, red-black) are designed to prevent this.

2.3.3 Sorting Algorithms

For each sort we cover: the idea, pseudocode, a worked example, best/average/worst time, space, **stability** (does equal-key order survive?), and **adaptivity** (does it run faster on nearly-sorted data?).

Bubble Sort

Bubble sort makes repeated passes through a list, comparing each adjacent pair and swapping them if they are out of order. Larger elements "bubble up" to the end of the list one position per pass.

```

PROCEDURE bubbleSort(list)
    n := length(list)
    FOR i := 0 TO n - 2

```

```

swapped := FALSE
FOR j := 0 TO n - 2 - i      // last i items already sorted
    IF list[j] > list[j + 1] THEN
        swap(list[j], list[j + 1])
        swapped := TRUE
    ENDIF
ENDFOR
IF NOT swapped THEN
    RETURN                  // early termination – already sorted
ENDIF
ENDFOR
ENDPROCEDURE

```

Worked example — one full pass through [5, 2, 8, 1, 9]

Compare	Before	Swap?	After
5 vs 2	[5 , 2, 8, 1, 9]	Yes (5 > 2)	[2, 5, 8, 1, 9]
5 vs 8	[2, 5 , 8, 1, 9]	No	[2, 5, 8, 1, 9]
8 vs 1	[2, 5, 8 , 1, 9]	Yes (8 > 1)	[2, 5, 1, 8, 9]
8 vs 9	[2, 5, 1, 8 , 9]	No	[2, 5, 1, 8, 9]

After one pass the largest value (9) is in its final position. Subsequent passes will bubble 8 into position, then 5, and so on.

Bubble Sort — one full pass through [5, 2, 8, 1, 9]



Property	Value
Best case	$O(n)$ — sorted input, one pass, flag catches it
Average	$O(n^2)$
Worst	$O(n^2)$ — reverse-sorted input
Space	$O(1)$ in-place
Stable?	Yes — only swaps strictly out-of-order pairs
Adaptive?	Yes (with early-termination flag)

When to use: teaching, very small or nearly-sorted data. In practice bubble sort is almost never the right choice in production software.

Insertion Sort

Insertion sort builds a sorted sub-array at the start of the list. For each subsequent element it shuffles that element leftwards past larger values until it sits in the correct position of the

sorted prefix.

```

PROCEDURE insertionSort(list)
  FOR i := 1 TO length(list) - 1
    key := list[i]
    j := i - 1
    WHILE j ≥ 0 AND list[j] > key
      list[j + 1] := list[j]  // shift right
      j := j - 1
    ENDWHILE
    list[j + 1] := key        // insert into gap
  ENDFOR
ENDPROCEDURE

```

Worked example — sort [5, 2, 8, 1, 9]

Pass	key	Sorted prefix before	Array after insert
i=1	2	[5]	[2, 5, 8, 1, 9]
i=2	8	[2, 5]	[2, 5, 8, 1, 9]
i=3	1	[2, 5, 8]	[1, 2, 5, 8, 9]
i=4	9	[1, 2, 5, 8]	[1, 2, 5, 8, 9]

Insertion Sort — [5, 2, 8, 1, 9]

Start	52819	key=2, insert before 5
Pass 1	25819	key=8, already in place
Pass 2	25819	key=1, shift 2,5,8 right
Pass 3	12589	key=9, already in place
Done	12589	sorted

Green = sorted prefix; yellow = key being inserted

Property	Value
Best case	$O(n)$ — already sorted; inner loop never runs
Average	$O(n^2)$
Worst	$O(n^2)$ — reverse-sorted
Space	$O(1)$ in-place
Stable?	Yes
Adaptive?	Yes — very fast on nearly-sorted data

When to use: small data (up to ~20 elements), nearly sorted data, online sorting (each new element arrives one at a time). Production sort libraries often fall back to insertion sort for small sub-arrays inside quick sort or merge sort.

Merge Sort

Merge sort is a **divide-and-conquer** algorithm: the list is recursively split in half until sub-lists contain one element; pairs of sorted sub-lists are then merged back together to produce longer sorted sub-lists.

```
FUNCTION mergeSort(list)
  IF length(list) ≤ 1 THEN
    RETURN list                // base case – already sorted
  ENDIF
  mid := length(list) DIV 2
  left := mergeSort(list[0 .. mid - 1])
  right := mergeSort(list[mid .. length(list) - 1])
  RETURN merge(left, right)
ENDFUNCTION

FUNCTION merge(left, right)
  result := []
  i := 0
```

```

j := 0
WHILE i < length(left) AND j < length(right)
  IF left[i] ≤ right[j] THEN
    append result, left[i]
    i := i + 1
  ELSE
    append result, right[j]
    j := j + 1
  ENDIF
ENDWHILE
append result, remaining elements of left
append result, remaining elements of right
RETURN result
ENDFUNCTION

```

Why $O(n \log n)$?

Splitting halves the sub-list size each recursion, giving $\log_2 n$ levels of recursion. At *each* level the merge step does $O(n)$ work in total (the sub-lists at a level together cover all n elements). Total: $n \times \log n$.

Worked example — sort [5, 2, 8, 1, 9, 3, 7, 4]

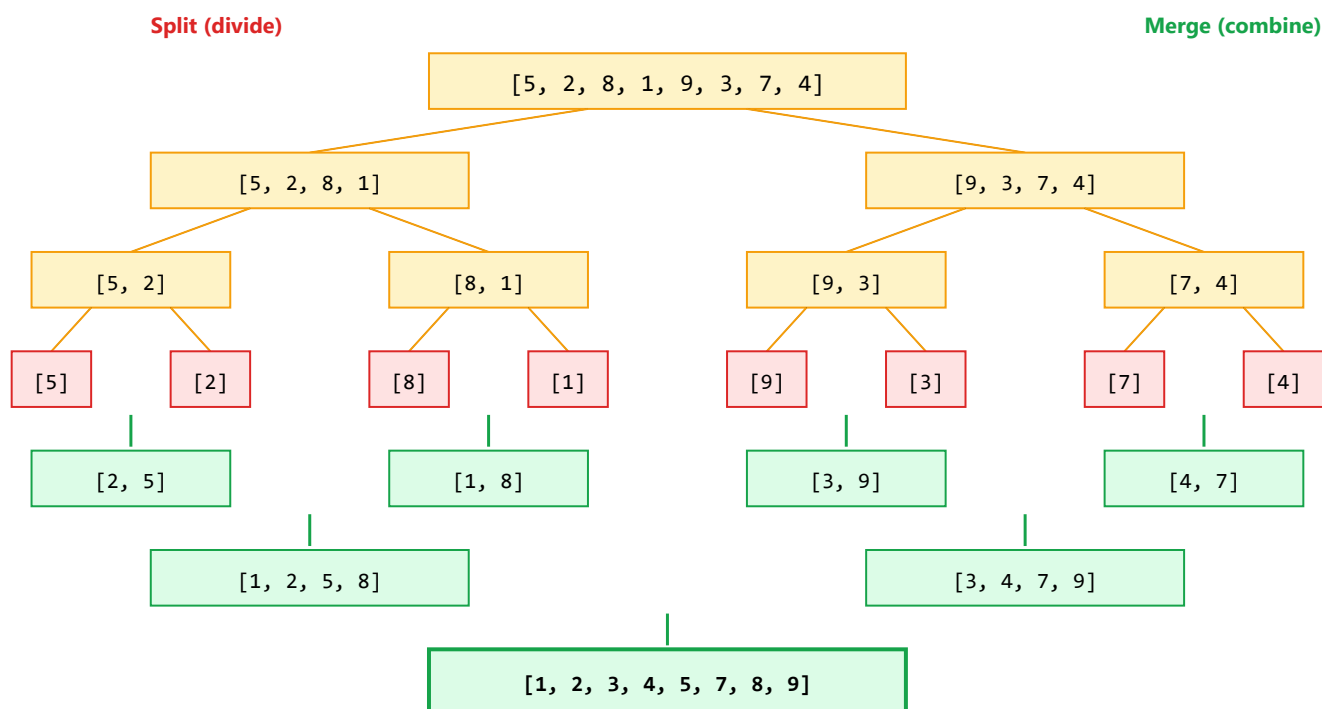
Recursion tree (split phase):

- [5, 2, 8, 1, 9, 3, 7, 4]
- [5, 2, 8, 1] and [9, 3, 7, 4]
- [5, 2], [8, 1], [9, 3], [7, 4]
- [5], [2], [8], [1], [9], [3], [7], [4]

Merge phase (bottom-up):

- [5]+[2]→[2,5]; [8]+[1]→[1,8]; [9]+[3]→[3,9]; [7]+[4]→[4,7]
- [2,5]+[1,8]→[1,2,5,8]; [3,9]+[4,7]→[3,4,7,9]
- [1,2,5,8]+[3,4,7,9]→[1,2,3,4,5,7,8,9]

Merge Sort — recursion + merge



$\log_2 8 = 3 \text{ levels of merging} \times O(n) \text{ work per level} = O(n \log n)$

Property	Value
Best case	$O(n \log n)$
Average	$O(n \log n)$
Worst	$O(n \log n)$
Space	$O(n)$ auxiliary — not in-place
Stable?	Yes
Adaptive?	No — does the same work regardless of input

When to use: large data sets where a guaranteed $O(n \log n)$ worst case matters (avionics, financial systems); external sorting (data that does not fit in RAM) — merges work beautifully with streaming; linked lists (no random access needed).

Exam Tip: Merge sort trades **$O(n)$ extra memory** for its guaranteed $O(n \log n)$ bound. If memory is tight, prefer quick sort (in-place).

Why merge sort is stable

During the merge step, the comparison `left[i] ≤ right[j]` (note the " $\leq$ ", not " $<$ ") prefers the left element when values are equal. Because the left sub-list came from earlier in the original array, equal keys keep their original relative order. This is exactly what *stability* means — and it is why many database and spreadsheet sorts are based on merge sort, since users often sort by one column and then another, relying on the earlier sort to survive the later one.

External sorting

"External" sorting means sorting data that is too large to fit in main memory, e.g. a 500 GB log file on a machine with 16 GB of RAM. Merge sort is the algorithm of choice: split the file into chunks each small enough to fit in RAM, sort each chunk with an in-memory sort, write back to disk, then *k-way merge* the sorted chunks by streaming from each file and repeatedly picking the smallest current element. This minimises disk seeks and respects the streaming nature of the data.

Quick Sort

Quick sort is a divide-and-conquer algorithm. A **pivot** element is chosen and the list is **partitioned** into elements $<$ pivot, the pivot itself, and elements $\geq$ pivot. Quick sort is then applied recursively to the two sub-lists.

Lomuto partition scheme

```
FUNCTION quickSort(list, low, high)
  IF low < high THEN
    p := partition(list, low, high)
    quickSort(list, low, p - 1)
    quickSort(list, p + 1, high)
  ENDIF
ENDFUNCTION

FUNCTION partition(list, low, high)
  pivot := list[high]           // Lomuto: pivot = last element
  i := low - 1
  FOR j := low TO high - 1
```

```

    IF list[j] ≤ pivot THEN
        i := i + 1
        swap(list[i], list[j])
    ENDIF
ENDFOR
swap(list[i + 1], list[high])    // place pivot in final position
RETURN i + 1
ENDFUNCTION

```

Hoare scheme (briefly): two pointers move inward from opposite ends, swapping elements they find on the wrong side of the pivot until they cross. Hoare does fewer swaps on average but is trickier to code correctly.

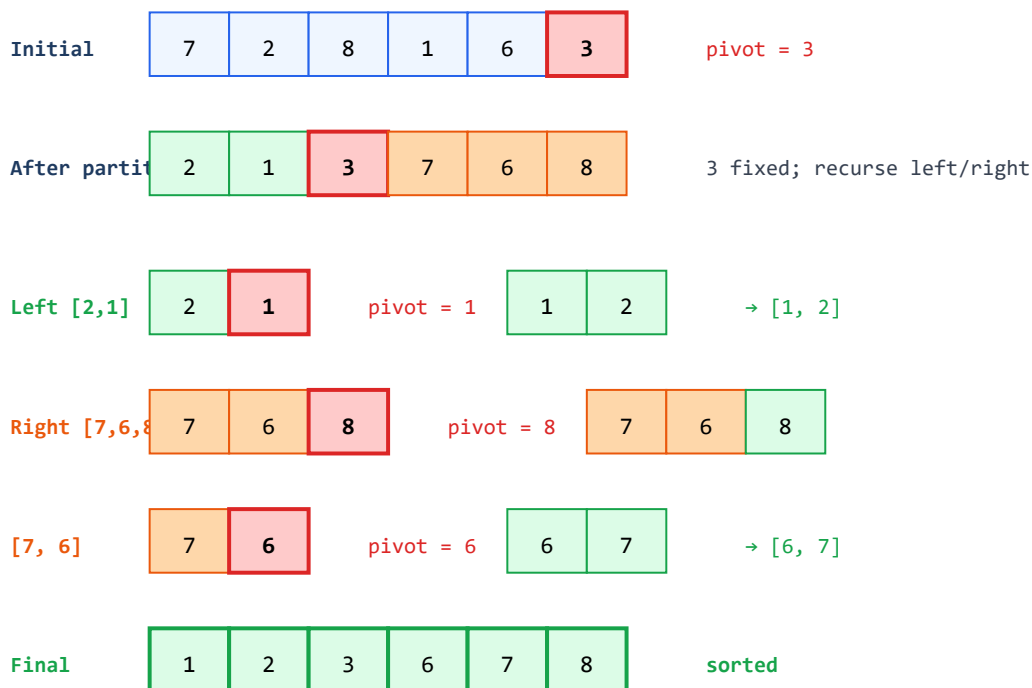
Pivot selection strategies

- **First or last element** — simplest; degenerates to $O(n^2)$ on already-sorted input.
- **Random element** — expected $O(n \log n)$ regardless of input shape; eliminates adversarial inputs.
- **Median-of-three** — pick the median of first, middle, last element. Cheap, usually good, used in most production implementations (e.g. C++ `std::sort`).
- **Middle element** — simple and works well on sorted input.

Worked example — sort [7, 2, 8, 1, 6, 3]; pivot = last

Step	List	Pivot	Result
1	[7, 2, 8, 1, 6, 3]	3	Partition → [2, 1, 3, 7, 6, 8]; pivot index 2
2a	Left: [2, 1]	1	Partition → [1, 2]; pivot index 0
2b	Right: [7, 6, 8]	8	Partition → [7, 6, 8]; pivot index 2
3	[7, 6]	6	Partition → [6, 7]
Final	[1, 2, 3, 6, 7, 8]		

Quick Sort — partition with Lomuto (pivot = last)



Property	Value
Best case	$O(n \log n)$ — balanced partitions
Average	$O(n \log n)$
Worst	$O(n^2)$ — sorted input with first/last pivot
Space	$O(\log n)$ recursion stack (average)
Stable?	No — swaps can cross equal keys
Adaptive?	Not naturally; worst case occurs on sorted data with naive pivot

Exam Tip: If you are asked *why* quick sort is usually preferred over merge sort in practice, mention: (1) in-place — only $O(\log n)$ space, (2) better cache locality from contiguous swaps, (3) smaller hidden constants. Merge sort wins only when worst-case guarantees or stability are required.

Why the $O(n^2)$ worst case happens

If the pivot is always the smallest (or largest) element of the current sub-array — which happens with a "first element" or "last element" pivot when the input is already sorted or reverse-sorted — then each partition shaves just one element off the sub-array. Instead of halving the work at every step we decrement by one, giving a recursion depth of n rather than $\log n$. Total work becomes $n + (n-1) + (n-2) + \dots + 1 = n(n+1)/2$, i.e. $O(n^2)$.

Randomised pivot selection or median-of-three makes this worst case vanishingly unlikely.

Production libraries such as the C++ STL's `std::sort` use **introsort**, which starts as quick sort but falls back to heap sort if recursion goes too deep — giving the best of both worlds (quick sort's speed with an $O(n \log n)$ worst-case guarantee).

Tail-call optimisation and stack depth

A naive recursive implementation of quick sort can use $O(n)$ stack space in the worst case. Production implementations recurse only on the smaller sub-array and iterate over the larger one (or use *tail-call optimisation* where available). This bounds the stack depth at $O(\log n)$ even in adverse cases.

Sorting Comparison Summary

Algorithm	Best	Average	Worst	Space	Stable?	In-place?	Best use case
Bubble sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes	Teaching; tiny / nearly-sorted lists
Insertion sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes	Small (<20) or nearly-sorted data; online sort
Merge sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	No	Large data; guaranteed worst-case; linked lists; external sort
Quick sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No	Yes	General-purpose in-memory sorting

Choosing the Right Sort

A surprisingly common exam question gives you a scenario and asks which sort you would choose. A good way to reason:

Scenario	Best choice	Reason
Teacher demonstrating sorting in a Year 7 classroom	Bubble sort	Easy to explain; visually obvious.
Sorting 12 numbers from a dice-game	Insertion sort	Simple, fast on tiny arrays, no hidden constants.
Sorting a million records in memory on a desktop PC	Quick sort (or introsort)	Fast average case, in-place, cache-friendly.
Sorting a 500 GB file on a server with 16 GB of RAM	External merge sort	Streams data, minimises disk seeks.
Sorting by surname then by forename (preserving surname ties)	Merge sort (or any stable sort)	Stability preserves the first-pass order on ties.
Embedded system with 2 KB RAM, small data	Insertion sort	$O(1)$ space and simple machine code footprint.
Safety-critical avionics where a worst-case guarantee is mandatory	Merge sort or heap sort	Deterministic $O(n \log n)$ worst case.

2.3.4 Graph Algorithms

Graph Recap

(See 1.4.2 for internals.) A **graph** is a set of *vertices* (nodes) connected by *edges*. Edges can be **directed** or **undirected**, and **weighted** (edges carry a cost, e.g. distance, travel time, bandwidth). Two common representations:

- **Adjacency matrix** — a $V \times V$ grid; fast edge lookup, $O(V^2)$ space even if sparse.

- **Adjacency list** — each vertex stores a list of neighbours; space-efficient for sparse graphs.

Traversal algorithms (**BFS**, **DFS**) visit every reachable vertex in a specific order, running in $O(V + E)$ time. OCR treats these in 1.4; here we focus on **weighted-graph** shortest-path algorithms.

Dijkstra's Shortest-Path Algorithm

Dijkstra's algorithm (1956) finds the shortest path from a single *source* vertex to every other vertex in a graph with **non-negative** edge weights. It is a greedy algorithm that repeatedly extracts the unvisited vertex with the smallest known distance and relaxes all its outgoing edges.

Exam Tip: Dijkstra's algorithm **requires non-negative edge weights**. If negative weights are present, use **Bellman–Ford** instead. This is a favourite exam gotcha.

The algorithm

1. Assign every vertex a tentative distance: 0 for the source, ∞ for every other vertex.
2. Mark all vertices unvisited; put the source into a priority queue keyed by distance.
3. Loop while the priority queue is non-empty:
 1. Extract the vertex `u` with the smallest tentative distance.
 2. Mark `u` as visited (its distance is now final).
 3. For each unvisited neighbour `v` of `u`:
 - Compute `alt = dist[u] + weight(u, v)`.
 - If `alt < dist[v]`, set `dist[v] := alt`, set `prev[v] := u`, and add/update `v` in the priority queue. This step is called **relaxation**.
4. When the queue is empty, `dist[]` holds the shortest distance from source to every vertex, and `prev[]` lets you reconstruct the path.

```
FUNCTION dijkstra(graph, source)
  FOR each vertex v IN graph
    dist[v] := INFINITY
    prev[v] := UNDEFINED
  ENDFOR
```

```

dist[source] := 0
pq := new PriorityQueue()
pq.insert(source, 0)

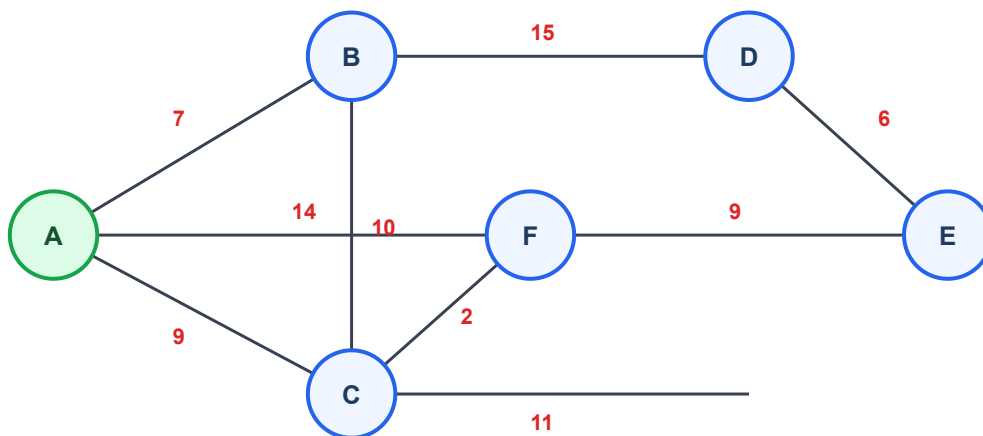
WHILE pq is not empty
  u := pq.extractMin()
  FOR each neighbour v OF u
    alt := dist[u] + weight(u, v)
    IF alt < dist[v] THEN
      dist[v] := alt
      prev[v] := u
      pq.insertOrDecreaseKey(v, alt)
    ENDIF
  ENDFOR
ENDWHILE
RETURN dist, prev
ENDFUNCTION

```

Worked example — 6-node weighted graph, source = A

Edges (undirected, non-negative weights): A–B=7, A–C=9, A–F=14, B–C=10, B–D=15, C–D=11, C–F=2, D–E=6, E–F=9.

Dijkstra's Algorithm — 6-node weighted graph



Green A = source. Red numbers are edge weights.

Step-by-step distance table (∞ means "no path found yet"; bold = distance now finalised):

Iter.	Visited vertex	A	B	C	D	E	F	Priority queue after step

0	— init	0	∞	∞	∞	∞	∞	{A:0}
1	A (dist 0)	0	7	9	∞	∞	14	{B:7, C:9, F:14}
2	B (dist 7)	0	7	9	22	∞	14	{C:9, F:14, D:22}
3	C (dist 9)	0	7	9	20	∞	11	{F:11, D:20}
4	F (dist 11)	0	7	9	20	20	11	{D:20, E:20}
5	D (dist 20)	0	7	9	20	20	11	{E:20}
6	E (dist 20)	0	7	9	20	20	11	{ } — done

Shortest distances from A: A=0, B=7, C=9, F=11, D=20, E=20. Shortest path A→E follows `prev[]` : E←F←C←A, length 9+2+9 = 20.

Complexity

Implementation	Time complexity
Array / scan for min	$O(V^2)$
Binary heap (typical)	$O((V + E) \log V)$
Fibonacci heap	$O(E + V \log V)$

Reconstructing the Shortest Path

Dijkstra produces a `prev[]` array where `prev[v]` is the predecessor of `v` on the best-known path from source. To recover the actual vertex sequence, start at the destination and follow `prev` pointers back to the source, then reverse.

```

FUNCTION reconstructPath(prev, destination)
    path := []
    current := destination
    WHILE current != UNDEFINED
        prepend path, current
        current := prev[current]
```

```

    ENDWHILE
    RETURN path
ENDFUNCTION

```

In the worked example above, starting from E: $\text{prev}[E]=F$, $\text{prev}[F]=C$, $\text{prev}[C]=A$, $\text{prev}[A]=\text{undefined}$.
 Reversed: $A \rightarrow C \rightarrow F \rightarrow E$, total distance 20.

A* (A-Star) Algorithm

A* is an informed search algorithm that extends Dijkstra with a **heuristic** $h(n)$ that estimates the remaining cost from node n to the goal. Instead of selecting the node with the smallest $g(n)$, it selects the one with the smallest $f(n) = g(n) + h(n)$. A* directs search towards the goal rather than exploring outward uniformly.

- $g(n)$ — the known cost of the shortest path from start to n .
- $h(n)$ — the estimated remaining cost from n to the goal (problem-specific).
- $f(n) = g(n) + h(n)$ — the estimated total path cost through n .

Heuristic properties

- **Admissible** — $h(n)$ *never overestimates* the true remaining cost. Admissibility guarantees A* returns an **optimal** path.
- **Consistent (monotonic)** — satisfies the triangle inequality: for every edge from n to m , $h(n) \leq \text{weight}(n,m) + h(m)$. Consistency implies admissibility and lets A* settle each node's g the first time it is expanded (no re-expansion needed).

Common heuristics

- **Straight-line (Euclidean) distance** — $\sqrt{(x_1-x_2)^2 + (y_1-y_2)^2}$. Admissible in continuous 2D space.
- **Manhattan distance** — $|x_1-x_2| + |y_1-y_2|$. Admissible on a grid where you can only move up/down/left/right.
- **Chebyshev distance** — $\max(|dx|, |dy|)$. Admissible when diagonal moves cost the same as orthogonal.
- **Zero heuristic** — $h(n) = 0$ for all n . Always admissible; reduces A* exactly to Dijkstra.

Exam Tip: The standard marking point is: "When $h(n) = 0$, A* is equivalent to Dijkstra's algorithm." If the heuristic is a poor estimate (large overestimate) A* can return a sub-optimal path.

Pseudocode

```

FUNCTION aStar(graph, start, goal, h)
    openSet := PriorityQueue()
    openSet.insert(start, h(start))
    gScore[start] := 0
    fScore[start] := h(start)

    WHILE openSet is not empty
        current := openSet.extractMin()           // lowest f-score
        IF current = goal THEN
            RETURN reconstructPath(prev, current)
        ENDIF
        FOR each neighbour n OF current
            tentativeG := gScore[current] + weight(current, n)
            IF tentativeG < gScore[n] THEN
                prev[n] := current
                gScore[n] := tentativeG
                fScore[n] := tentativeG + h(n)
                IF n NOT IN openSet THEN
                    openSet.insert(n, fScore[n])
                ENDIF
            ENDIF
        ENDFOR
    ENDWHILE
    RETURN FAILURE                               // no path
ENDFUNCTION

```

Worked example — same 6-node graph with heuristic values

Suppose the goal is **E** and the heuristic (straight-line estimates) are:

Node	A	B	C	D	E	F
$h(n)$	20	17	11	6	0	9

Expansion order (lowest $f = g + h$ first):

Step	Expand	Reasoning
1	A	$f(A) = 0 + 20 = 20$. Relax neighbours: B ($f=7+17=24$), C ($f=9+11=20$), F ($f=14+9=23$).
2	C	Lowest $f=20$. Relax: D via C $f=(9+11)+6=26$, F via C $f=(9+2)+9=20$.
3	F	Lowest $f=20$. Relax: E via F $f=(11+9)+0=20$.
4	E	E = goal — return path $A \rightarrow C \rightarrow F \rightarrow E$, cost 20.

Notice A* never expanded B or D because better options were always at the front of the open set — the heuristic *pruned* those branches. Dijkstra, with no heuristic, would have expanded B before finding E.

A* on a grid — Manhattan heuristic, wall in middle



A* prefers cells with $f=6$ along the direct route; the $f=8$ detour is only considered if blocked.

When to use A*

- **Video games** — pathfinding for NPCs and units on tile/grid maps.
- **Robotics** — motion planning through obstacle fields.
- **GPS / satnav** — routing through road networks with distance/time heuristics.

- **Puzzle solving** — 8-puzzle, 15-puzzle (heuristic: Manhattan distance of each tile from its goal position).

Trade-offs of A*

- **Memory** — A* stores every expanded node in the open and closed sets; this can balloon in large search spaces. Memory-bounded variants (IDA*, SMA*) exist.
- **Heuristic quality** — a tight, accurate admissible heuristic dramatically reduces the number of expansions. A loose heuristic (e.g. $h = 0$) makes A* no better than Dijkstra.
- **Inadmissible heuristics** — sometimes faster but may return a sub-optimal path. Games often use a scaled-up heuristic ($h' = 1.5 h$) to trade off a little optimality for much faster searches.
- **Dynamic environments** — if edge costs change (e.g. traffic updates), the path must be re-planned. **D*** (dynamic A*) and **D* Lite** incrementally repair the existing solution rather than re-running from scratch.

Dijkstra vs A* Summary

Aspect	Dijkstra	A*
Heuristic used?	No — uniform-cost	Yes — $h(n)$ estimates cost to goal
Finds	Shortest paths to <i>every</i> vertex	Shortest path to a <i>single</i> goal
Optimal?	Yes (if weights ≥ 0)	Yes (if heuristic is admissible)
Speed	Explores outward in all directions	Usually faster — heuristic directs search
Typical use	Network routing, all-pairs pre-computation	Games, robotics, GPS single-destination routing
Negative edges?	No — needs non-negative weights	No

2.3.5 OCR Pseudocode Conventions

OCR exam pseudocode follows a fairly consistent style. You do *not* lose marks for slight syntax variations as long as your intent is clear and the code runs on paper.

- **Assignment:** `x := 5` or `x = 5` .
- **Indentation:** used to show block structure.
- **Selection:** `IF condition THEN ... ELSEIF ... ELSE ... ENDIF` .
- **Count loop:** `FOR i := 0 TO n - 1 ... ENDFOR` .
- **Condition loop:** `WHILE condition ... ENDWHILE` ; or `REPEAT ... UNTIL condition` .
- **Subroutines:** `PROCEDURE name(params) ... ENDPROCEDURE` ; `FUNCTION name(params) ... RETURN value ... ENDFUNCTION` .
- **Integer division:** `DIV` ; **remainder:** `MOD` .
- **Output / input:** `print(x)` , `input()` .
- **Arrays:** zero-indexed, e.g. `list[0]` , `length(list)` .

A small worked template

```
PROCEDURE printEvens(n)
  FOR i := 0 TO n
    IF i MOD 2 = 0 THEN
      print(i)
    ENDIF
  ENDFOR
ENDPROCEDURE
```

Worked Mini-Questions

Q1. State the time complexity of the following algorithm in Big-O notation, and justify.

```
FUNCTION mystery(n)
  total := 0
  FOR i := 1 TO n
    FOR j := i TO n
      total := total + 1
```

```
        ENDFOR
    ENDFOR
    RETURN total
ENDFUNCTION
```

Answer: The inner loop runs n times when $i = 1$, $n-1$ times when $i = 2$, ..., 1 time when $i = n$. Total iterations = $n + (n-1) + \dots + 1 = n(n+1)/2$. Dropping constants and lower-order terms: $O(n^2)$.

Q2. An unsorted list of 1 024 integers is searched for a value. On average, how many comparisons does a linear search make? What about binary search?

Answer: Linear search performs on average about $n/2 = 512$ comparisons. Binary search cannot be used on an unsorted list — it would have to be sorted first ($O(n \log n) = \sim 10\,000$ operations) before achieving $\log_2(1024) = 10$ comparisons per search. Binary search only pays off if many searches are performed against the same list.

Q3. Name one scenario where quick sort's worst case occurs and suggest a mitigation.

Answer: Sorting an already-sorted (or reverse-sorted) list with a first-element or last-element pivot produces maximally unbalanced partitions, giving $O(n^2)$. Mitigations: random pivot, median-of-three pivot, or switching to introsort (quick sort + heap sort fallback).

Q4. Explain briefly why A* returns an optimal path when the heuristic is admissible.

Answer: An admissible heuristic never overestimates the true remaining cost, so $f(n) = g(n) + h(n)$ is always a lower bound on the cost of any path through n . A* expands the node with the smallest f -value first; the first time it expands the goal, no unexplored node can possibly improve on that f -value, so the path found is optimal.

Q5. Trace one pass of bubble sort on [4, 1, 3, 2].

Answer:

- Compare 4 vs 1 → swap → [1, 4, 3, 2]

- Compare 4 vs 3 → swap → [1, 3, 4, 2]
- Compare 4 vs 2 → swap → [1, 3, 2, 4]

After one pass the maximum (4) is in its final position. The list still requires more passes to finish (swap 3 and 2).

Exam Preparation — Priority Checklist

Focus your revision on these high-frequency items:

- **Big-O ordering and examples** — be ready to rank algorithms and justify with one sentence.
- **Binary search vs linear search** — always state the *sorted* precondition for binary search.
- **Trace a sort by hand** — practise writing out each pass/step of bubble, insertion, merge and quick sort on arrays of length 5–8.
- **Sorting comparison table** — memorise best/average/worst for all four sorts; memorise stable/in-place status.
- **Dijkstra trace** — draw the distance table and update it row-by-row. State that weights must be non-negative.
- **A* essentials** — $f = g + h$; admissible heuristic; $A^* = \text{Dijkstra}$ when $h = 0$; applications (games, GPS, robotics).
- **Pseudocode** — be fluent enough to write a binary search, a bubble-sort pass and an insertion sort without the slide in front of you.

Exam Tip: Classic long-answer question: "Compare merge sort and quick sort. Explain which you would use and why." A great answer contrasts *guarantees* (merge sort: $O(n \log n)$ always; quick sort: $O(n^2)$ worst), *memory* (merge: $O(n)$; quick: $O(\log n)$ in-place), *stability* (merge yes, quick no), then justifies by scenario (embedded: quick; safety-critical: merge; external storage: merge).

Common Pitfalls and How to Avoid Them

Pitfall	Fix
Saying binary search works on "any list"	Always state the list must be sorted .
Quoting big-O with constants (e.g. "O(2n)")	Drop constants — write O(n).
Forgetting the early-termination flag in bubble sort best case	Write the flag; state that O(n) best case depends on it.
Confusing stability with in-place	Stability = equal keys keep their relative order; in-place = O(1) auxiliary space. Independent properties.
Claiming Dijkstra works on any graph	Add: edge weights must be non-negative.
Forgetting $f = g + h$ in A*	Always define both terms: g = known cost from start; h = estimated cost to goal.
Saying merge sort is O(n)	It is O(n log n). O(n) refers to its <i>space</i> .
Claiming quick sort is always faster than merge sort	On average, yes; worst case it is O(n ²). Mention both.

Cross-Topic Links

- **1.4 Data structures** — sorted arrays enable binary search; binary search trees offer O(log n) insert/search when balanced; priority queues (binary heaps) underpin Dijkstra and A*.
- **2.1 Computational thinking** — divide-and-conquer (merge and quick sort) and abstraction (choosing a heuristic in A*) are core computational-thinking patterns.
- **2.2 Programming techniques** — recursion appears in merge sort, quick sort, binary tree search and the recursive form of binary search.
- **1.3 Exchanging data** — searching and sorting are fundamental to databases and networks; many SQL operations are essentially sorts and searches over indexed B-trees.

Quick-Reference — "Which algorithm do I reach for?"

Problem	Algorithm	Complexity
Find a value in an unsorted array	Linear search	$O(n)$
Find a value in a sorted array	Binary search	$O(\log n)$
Find a value in a balanced BST	Binary tree search	$O(\log n)$
Sort a small array	Insertion sort	$O(n^2)$, fast for small n
Sort a large array in memory	Quick sort (introsort)	$O(n \log n)$ avg
Sort with a guaranteed worst case	Merge sort / heap sort	$O(n \log n)$
Sort with stability required	Merge sort	$O(n \log n)$
Sort data too big to fit in RAM	External merge sort	$O(n \log n)$ with disk I/O
Shortest path from one vertex to all others	Dijkstra's algorithm	$O((V+E) \log V)$
Shortest path to a single goal with a heuristic	A*	$O((V+E) \log V)$, usually faster in practice
Graph with negative edges	Bellman–Ford	$O(V E)$

Glossary

Term	Meaning
Algorithm	Finite, unambiguous sequence of steps turning inputs into outputs.

Big-O	Asymptotic upper bound on growth of running time or space.
Stable sort	Preserves the relative order of equal keys.
In-place sort	Uses $O(1)$ or $O(\log n)$ auxiliary memory.
Adaptive sort	Runs faster on nearly-sorted input.
Relaxation	Step of updating a shorter known distance to a vertex.
Admissible heuristic	Never overestimates the true remaining cost.
Consistent heuristic	Satisfies the triangle inequality (implies admissibility).

Summary

- An algorithm is a finite, unambiguous procedure that transforms inputs into outputs.
- Big-O notation expresses asymptotic worst-case upper bounds on time or space.
- Linear search is $O(n)$ and works on any list; binary search is $O(\log n)$ but *requires sorted data*.
- Bubble and insertion sort are $O(n^2)$ quadratic sorts; merge sort guarantees $O(n \log n)$ at the cost of $O(n)$ extra memory; quick sort is $O(n \log n)$ on average, $O(n^2)$ worst, but in-place and usually fastest in practice.
- Dijkstra's algorithm finds shortest paths in graphs with non-negative weights by greedy relaxation.
- A* extends Dijkstra with a heuristic $h(n)$; when $h = 0$, A* reduces to Dijkstra. A* is widely used in games, robotics and GPS.
- Space/time trade-offs, stability and adaptivity are all legitimate criteria when choosing between algorithms.